

Day 4

8/3 월

오늘 서울 기온을 진짜로 받아오는 날

01

코드가 기다리지 않고 넘어간다는 것을 설명할 수 있다

02

async 와 await 로 순서대로 읽히는 코드를 쓸 수 있다

03

fetch 로 인터넷에서 데이터를 받아 화면에 그릴 수 있다

오늘의 흐름

교시마다 실습이 두 번씩 있음. 짧게 자주 칠 것

오전, 이론 09:00 - 12:50

■ 09:00	기다리지 않는 코드
■ 10:00	async 와 await
■ 11:00	fetch 로 진짜 데이터
■ 12:00	에러 처리와 화면 연결

오후, 실습 14:00 - 17:50

■ 14:00	미션 브리핑, 함께 작성
■ 15:00	개인 개발, 중간 풀이
■ 16:00	개인 개발, 중간 풀이
■ 17:00	추가 문제, 마무리

01 기다리지 않는 코드

지금까지 데이터는 어디서 왔나

전부 제가 파일에 적어 드린 것

Day 2 설문 30명. 파일 맨 위에 적혀 있었음

Day 3 메뉴 목록, 질문 목록. 이것도 파일 안에

Day 4 인터넷에 있는 진짜 데이터를 받아옴

지난 질문 진짜 서비스라면 이 데이터는 어디에 있을까. 오늘 그 답을 봅니다

오늘 도착할 곳

지금 이해 안 되는 게 정상

■ Python, 크롤링 때

이미 해본 것

```
import requests

response = requests.get(url)
data = response.json()

print(data)
```

■ JavaScript, 오늘

오전이 끝나면 읽힘

```
async function load() {
  const response = await fetch(url);
  const data = await response.json();

  console.log(data);
}
```

새로 나온 것 **async** 와 **await** 두 단어뿐. 나머지는 하는 일이 같다

같은 코드, 다른 순서

1초 기다렸다 찍기

■ Python

멈춰서 기다림

```
import time

print("1")
time.sleep(1)
print("2")
print("3")
```

1 2 3

■ JavaScript

안 기다리고 넘어감

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 1000);

console.log("3");
```

1 3 2

오늘의 출발점 자바스크립트는 오래 걸리는 일을 만나면 맡겨두고 다음 줄로 간다

순서를 맞혀보고 콘솔로 확인하기

```
console.log("A");  
setTimeout(() => console.log("B"), 2000);  
setTimeout(() => console.log("C"), 0);  
console.log("D");
```

할 일

먼저 종이에 순서를 적고, 그다음 콘솔에 붙여넣어 확인

생각해볼 것

0초짜리 C 는 왜 D 보다 늦게 나올까

남은 시간

08:00

왜 기다리지 않게 만들었나

브라우저는 화면을 그리는 일도 같이 하기 때문

파이썬 프로그램

터미널에서 혼자 돌아감. 1초 멈춰도 아무도 불편하지 않음

기다리는 동안 화면이 없으니 멈춰도 티가 안 남

브라우저

코드를 돌리는 일과 화면을 그리는 일을 한 사람이 함

코드가 멈추면 화면도 멈춤. 스크롤도 클릭도 안 먹음

그래서 오래 걸리는 일은 맡겨두고, 끝나면 알려 달라고 하는 방식을 씁니다

멈추면 어떻게 되는지 직접 보기

콘솔에 붙여넣고 3초

```
1 // 3초 동안 아무것도 안 하고 버티기
2 const end = Date.now() + 3000;
3 while (Date.now() < end) {}
4 console.log("끝");
5
6 // 실행하는 동안 해볼 것
7 //   페이지를 스크롤해보기
8 //   아무 버튼이나 눌러보기
9 //   글자를 드래그해보기
```

3초 동안
브라우저가 완전히
멈춥니다

이것이 이유 인터넷에서 데이터 받는 데 3초가 걸린다면, 그동안 화면이 이렇게 됩니다

맡겨두고 넘어간다

어제 배운 이벤트와 똑같은 구조

어제, 이벤트

```
button.addEventListener("click",
  () => {
    console.log("눌렀다");
  }
);
```

// 눌러면 실행해 주세요

오늘, 시간

```
setTimeout(
  () => {
    console.log("1초 지났다");
  },
  1000
);
// 1초 뒤에 실행해 주세요
```

같은 형태 나중에 할 일을 함수로 맡겨둔다. 어제 이미 해본 방식

3초 뒤에 제목 바꾸기

```
// practice.html 에서

// 1. 페이지를 열고 3초 뒤에 제목이 바뀌게
// 2. 그동안 아래 상자에 불러오는 중 이라고 표시
// 3. 바뀐 뒤에는 그 문구를 지우기
```

힌트 01

불러오는 중 표시는 setTimeout 밖에, 지우는 것은 안에

힌트 02

이게 오후 미션의 로딩 표시와 똑같은 구조

남은 시간

08:00

순서대로 해야 한다면

말기고 또 말기고

```
// 1초 뒤에 A, 그다음 1초 뒤에 B, 그다음 1초 뒤에 C
```

```
setTimeout(() => {  
  console.log("A");
```

```
  setTimeout(() => {  
    console.log("B");
```

```
      setTimeout(() => {  
        console.log("C");  
      }, 1000);  
    }, 1000);  
  }, 1000);
```

그래서 다음 시간에 이걸 세 줄로 만드는 방법을 배웁니다

02 **async 와 await**

Promise

지금은 없지만 나중에 준다는 약속

- 오래 걸리는 일을 시키면 결과 대신 약속을 돌려줌
- 아직 값이 없음. 나중에 채워짐
- 잘 되면 값을 주고, 안 되면 이유를 줌
- 오늘 우리가 만들 일은 없음. 받아서 쓰기만 함

대기
아직 안 끝남. pending

성공
값이 들어옴. fulfilled

실패
이유가 들어옴. rejected

약속을 만드는 쪽

외우지 말고 눈에만

```
// practice.html 에 들어 있는 함수

function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

// 이제 이렇게 쓸 수 있음
await wait(1000);
```

resolve

이걸 부르면 약속이 지켜짐. 끝났다고 알리는 신호

오늘 만들 일은 없음

fetch 처럼 이미 만들어진 것을 받아 쓰기만 함

왜 보여주나 AI 가 준 코드에 이 형태가 나오면 아 이거구나 하시라고

async 와 await

파이썬처럼 읽히게 만드는 두 단어

말기는 방식, 읽기 어려움

```
wait(1000).then(() => {  
  console.log("A");  
  return wait(1000);  
}).then(() => {  
  console.log("B");  
});
```

await 방식, 위에서 아래로

```
async function run() {  
  await wait(1000);  
  console.log("A");  
  
  await wait(1000);  
  console.log("B");  
}
```

await

이 약속이 끝날 때까지 여기서 기다림

async

이 함수 안에 기다림이 있다는 표시

규칙은 하나뿐

이것만 외우면 됨

`await` 는 `async` 함수 안에서만 쓸 수 있다

틀림

```
function load() {  
  const data = await fetch(url);  
}
```

// SyntaxError

맞음

```
async function load() {  
  const data = await fetch(url);  
}
```

// async 를 붙이면 됨

막히면 `await` 를 썼는데 에러가 나면 함수 앞에 `async` 가 있는지 보세요

중첩된 코드를 펴기

```
// practice.html 에 wait 함수가 준비되어 있습니다
//   await wait(1000) 하면 1초 기다립니다

// 1초 뒤 A, 다시 1초 뒤 B, 다시 1초 뒤 C 를
// async 함수 하나로 만들 것
```

힌트 01

12장 코드가 여섯 줄로 줄어듭니다

힌트 02

함수를 만들었으면 아래에서 한 번 불러줘야 실행됨

남은 시간

10:00

await 를 빼먹으면

오늘 가장 많이 날 실수

```
1 const data = getWeather();      // await 없음
2 console.log(data);
3 // Promise { <pending> }
4
5 data.temperature
6 // undefined 약속에는 그런 게 없음
7
8 const data = await getWeather();
9 data.temperature // 29.8 이제 나옴
```

값이 아니라
약속이 들어옴

await 하나 붙이면 끝

기억할 것 콘솔에 Promise 라고 찍히면 await 를 빼먹은 것입니다

일부러 틀려보기

```
// practice.html 에 getNumber 함수가 준비되어 있습니다
// 1초 뒤에 숫자 42 를 주는 async 함수입니다

// 1. await 없이 불러서 콘솔에 찍어보기
// 2. 거기에 더하기 1 을 해보기
// 3. await 를 붙여서 다시 해보기
```

확인할 것

2번에서 무엇이 나오는지. 에러가 나는지 아닌지

기억할 것

이 실수는 내일 디버깅 시간에 다시 만납니다

남은 시간

08:00

async 함수가 돌려주는 것 return 해도 값이 아님

```
1 async function getNumber() {  
2   return 42;  
3 }  
4  
5 getNumber()           // Promise, 42 가 아님  
6  
7 await getNumber()     // 42  
8  
9 // async 를 붙인 순간 반환값은 무조건 Promise
```

42 를 감싼
약속이 나옴

벗겨야 값이 나옴

그래서 내가 만든 async 함수를 부를 때도 앞에 await 가 필요합니다

then 방식도 읽을 줄 알아야

쓰지는 않아도 됨

■ then 방식

인터넷 예제에 많음

```
fetch(url)
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.log(error);
  });
```

■ await 방식

우리가 쓸 것

```
try {
  const response = await fetch(url);
  const data = await response.json();

  console.log(data);
} catch (error) {
  console.log(error);
}
```

대응 관계 then 하나가 await 한 줄. 마지막 catch 는 try catch 의 catch

여러 개를 받아야 할 때

순서대로 vs 동시에

하나씩, 느림

```
const seoul = await getWeather("서울");
const busan = await getWeather("부산");
const jeju = await getWeather("제주");
```

// 앞이 끝나야 다음이 시작

// 하나에 1초면 3초

한꺼번에, 빠름

```
const [seoul, busan, jeju] =
  await Promise.all([
    getWeather("서울"),
    getWeather("부산"),
    getWeather("제주")
  ]);
// 셋을 같이 보내고 다 오면 진행. 1초
```

쓸 때 서로 상관없는 요청이면 동시에. 앞 결과가 필요하면 하나씩

03 fetch 로 진짜 데이터

API, 데이터를 주는 창구

크롤링과 무엇이 다른가

크롤링, 봄에 한 것

사람이 보는 화면을 긁어서 필요한 부분만 뽑아냄

화면 구조가 바뀌면 코드가 깨짐

받는 것은 HTML 덩어리

API, 오늘 할 것

데이터를 주려고 만들어 둔 주소로 요청함

화면이 아니라 데이터만 옴. 깔끔함

받는 것은 JSON. 2일차에 배운 그것

오늘 쓸 곳 **Open-Meteo. 회원가입도 열쇠도 필요 없는 무료 날씨 API**

코드 치기 전에 주소부터

```
https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.978&t=temperature_2m
```

브라우저 주소창에 그대로 붙여넣고 엔터, 무엇이 나오는지 볼 것

주소 뜯어보기

물음표 뒤가 조건

```
https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.978&current=temperature_2m
```

주 소	어디에 물어볼지
물 음 표	여기서부터 조건이 시작된다는 표시
이 름 등 호 값	조건 하나. latitude 는 위도, longitude 는 경도
앰 퍼 샌 드	조건을 여러 개 이을 때. 어제 배운 그 기호와 모양만 같음

해 볼 것 위도를 35.18, 경도를 129.07 로 바꾸면 부산 날씨가 나옵니다

주소창만으로 데이터 바꿔보기

```
// 1. 내가 사는 도시의 날씨로 바꿔보기
//     위도 경도는 검색하면 나옵니다

// 2. current 에 습도를 추가하기
//     temperature_2m,relative_humidity_2m

// 3. 일부러 틀린 주소를 넣어보고 무엇이 오는지 보기
```

힌트 01

여러 값을 넣을 때는 쉼표로 잇습니다. 공백 없이

힌트 02

3번에서 오는 것도 JSON 입니다. 에러도 데이터로 옵니다

남은 시간

08:00

fetch

파이썬 requests 와 나란히

■ Python

이미 아는 것

```
import requests

response = requests.get(url)
data = response.json()

print(data["current"])
```

■ JavaScript

오늘 배우는 것

```
// 가져올 것 없음

const response = await fetch(url);
const data = await response.json();

console.log(data.current);
```

핵심 차이 줄 수가 같다. 앞에 await 가 두 번 붙은 것뿐

await 가 왜 두 번인가

기다리는 일이 두 개

1

`await fetch`

서버가 답을 보내줄 때까지 기다림. 상자가 도착하는 것

2

`await response.json`

받은 글자 덩어리를 객체로 바꿀 때까지 기다림. 상자를 여는 것

```
const response = await fetch(url);
console.log(response);          // Response 객체. 데이터가 아님
console.log(response.current);  // undefined
```

흔한 실수 두 번째 줄을 빼먹고 response 에서 값을 꺼내려다 undefined

진짜 기온 받아오기

콘솔에 한 줄씩

```
const url =  
  "https://api.open-meteo.com/v1/forecast" +  
  "?latitude=37.5665&longitude=126.978" +  
  "&current=temperature_2m";  
  
async function loadWeather() {  
  const response = await fetch(url);  
  const data = await response.json();  
  
  console.log(data);  
  console.log(data.current.temperature_2m);  
}  
  
loadWeather();
```

확인 지금 서울 기온이 숫자로 찍히면 성공입니다

Network 탭

주고받은 것을 눈으로 보는 곳

- F12 를 누르고 Network 탭. Console 옆에 있음
- 열어둔 상태에서 새로고침해야 기록이 남음
- 요청 하나를 클릭하면 주고받은 내용이 다 보임
- 코드가 아니라 통신 자체를 확인하는 곳

Status

200 이면 성공, 404 는 없는 주소, 500 은 서버 문제

Response

서버가 실제로 보낸 내용. JSON 이 그대로 보임

쓰는 때

코드가 문제인지 서버가 문제인지 가릴 때

받은 데이터 뜯어보기

current 와 hourly

current, 지금 한 시점

```
{
  current: {
    time: "2026-08-03T09:00",
    temperature_2m: 28.4
  }
}
```

```
// data.current.temperature_2m
```

hourly, 시간대별 목록

```
{
  hourly: {
    time: ["09:00", "10:00", ...],
    temperature_2m: [28.4, 29.1, ...]
  }
}
```

```
// 시각과 기온이 같은 번호로 대응
```

첫 번째로 할 일 모르는 API 를 만나면 data 를 통째로 콘솔에 찍고 구조부터 봅니다

API 문서에서 볼 곳은 세 군데

전부 읽을 필요 없음. 다음 주 프로젝트에 필요한 기술

1

주소

어디로 요청을 보내는지. 보통 문서 맨 위에 있음

2

넣을 수 있는 조건

Parameters 라고 적힌 표. 물음표 뒤에 붙일 것들

3

예제 응답

Response 나 Example. 무엇이 오는지 미리 봄

더 빠른 방법 문서에 있는 예제 주소를 브라우저에 그대로 넣어보는 것

프로젝트에 쓸 만한 API

가입도 열쇠도 필요 없는 것들

Open-Meteo	날씨. 오늘 쓰는 것	날씨 대시보드
공공데이터포털	지하철, 대기질, 문화행사. 가입은 필요	생활 정보
JSONPlaceholder	연습용 가짜 데이터. 글, 댓글, 사용자	연습
Nominatim	주소를 위도 경도로 바꿔줌	도시 검색

고를 때 주소를 브라우저에 넣어 JSON 이 나오면 쓸 수 있는 것입니다

받아온 기온을 화면에 그리기

```
// 1. 콘솔이 아니라 화면의 상자에 기온을 표시  
// 2. 버튼을 누르면 다시 불러오게  
// 3. 습도도 같이 받아서 함께 표시 (심화)
```

힌트 01

어제 배운 `textContent` 로 넣으면 됩니다

힌트 02

버튼 연결도 어제 배운 `addEventListener` 그대로

남은 시간

12:00

04 실패에 대비하기

오늘부터는 실패할 수 있습니다

내 컴퓨터 밖의 일에 기대기 시작했기 때문

인터넷이 끊김	fetch 자체가 실패. Failed to fetch
주소가 틀림	404 가 옴. 그런데 fetch 는 성공으로 침
서버가 아픔	500 이 옴. 내 잘못이 아님
데이터가 예상과 다름	키 이름이 다르거나 값이 비어 있음

그래서 잘 될 때만 생각하고 짜면 안 됩니다. 안 될 때를 같이 짜야 합니다

try 와 catch

파이썬의 try except

■ Python

이미 아는 것

```
try:
    response = requests.get(url)
    data = response.json()
except Exception as e:
    print(e)
```

■ JavaScript

이름만 다름

```
try {
    const response = await fetch(url);
    const data = await response.json();
} catch (error) {
    console.log(error.message);
}
```

핵심 차이 except 가 catch 로 바뀐 것뿐. 콜론 자리에 중괄호

try catch 만으로는 부족하다

404 는 안 잡힘

catch 로 가는 경우

서버에 아예 못 닿았을 때. 인터넷이 끊겼거나 주소 형식이 틀렸거나

Failed to fetch

catch 로 안 가는 경우

서버가 답은 줬는데 내용이 404 나 500 일 때

fetch 는 답을 받았으니 성공으로 칩니다

```
const response = await fetch(url);

// 그래서 직접 확인해야 함
if (!response.ok) {
  throw new Error("서버 응답 " + response.status);
}
```

`response.ok` 200 번대면 true, 아니면 false. throw 하면 catch 로 넘어갑니다

어느 쪽이든 해야 할 일 finally, 파이썬과 같음

두 곳에 똑같이 씀

```
try {  
    // ... 성공  
    status.textContent = "";  
} catch (error) {  
    // ... 실패  
    status.textContent = "";  
}
```

finally 로 한 번만

```
try {  
    // ... 성공  
} catch (error) {  
    // ... 실패  
} finally {  
    status.textContent = "";  
}
```

쓰는 자리 로딩 표시 지우기, 버튼 다시 켜기. 어느 쪽이든 해야 하는 일

기다리는 동안 화면은

사용자는 3초를 아주 길게 느낍니다

아무 표시가 없으면 고장난 줄 알고 버튼을 계속 누름

계속 누르면 요청이 여러 번 나가고 결과가 뒤섞임

그래서 두 가지 불러오는 중이라고 알리고, 버튼을 잠시 잠급니다

```
button.disabled = true;      // 시작할 때 잠그고  
// ...  
button.disabled = false;    // finally 에서 풀기
```

확인하는 법 로딩이 너무 빨라 안 보이면 앞에 `await wait(1000)` 을 잠깐 넣어볼 것

실패했을 때 뭐라고 쓸까

사용자와 개발자는 다른 것을 봐야 함

이렇게 쓰지 말 것

`TypeError: Failed to fetch`

사용자는 이게 무슨 말인지 모르고 뭘 해야 할지도 모름

이렇게

날씨를 불러오지 못했습니다. 다시 시도해 주세요

무엇이 안 됐는지와 무엇을 하면 되는지

```
} catch (error) {  
  console.log(error);           // 개발자용, 원본 그대로  
  status.textContent = "불러오지 못했습니다"; // 사용자용  
}
```

오늘 실습에서는 원인을 찾아야 하니 화면에도 `error.message` 를 같이 띄워도 됩니다

일부러 실패시켜 보기

```
// 1. 앞 실습 코드를 try catch 로 감싸기  
// 2. 주소에서 forecast 를 forecastX 로 바꿔보기  
//     catch 로 가는지, 아니면 그냥 넘어가는지 확인  
// 3. response.ok 확인을 넣어 화면에 안내 문구 띄우기
```

확인할 것

2번에서 Network 탭의 Status 가 몇으로 나오는지

힌트

console.log(response.ok) 를 찍어보면 바로 보입니다

남은 시간

10:00

브라우저가 막을 때 CORS

- 브라우저는 아무 서버에나 요청을 보내게 두지 않음
- 서버가 허용한다고 표시해 둔 곳만 됨
- 파이썬 requests 는 이 제한이 없음. 브라우저가 아니니까
- Open-Meteo 는 허용해 둔 곳이라 오늘은 잘 됨

이런 메시지를 보면
blocked by CORS policy

내 코드 문제가 아님
고칠 방법이 브라우저 쪽에는 없음

그래서 서버가 필요 내 서버가 대신 받아다 주면 됩니다. 그게 다음 달에 배울 Java 와 Spring

오늘의 완성형 오후 미션이 이 모양

```
async function loadWeather() {
  status.textContent = "불러오는 중";      // 1. 로딩 표시

  try {
    const response = await fetch(url);      // 2. 요청

    if (!response.ok) {                    // 3. 상태 확인
      throw new Error("응답 " + response.status);
    }

    const data = await response.json();    // 4. 해석
    render(data);                          // 5. 화면에
    status.textContent = "";
  } catch (error) {                       // 6. 실패하면
    status.textContent = "문제가 생겼습니다: " + error.message;
  }
}
```

render 는 어제 만든 그 함수입니다. 받아온 데이터로 화면을 그리기만 하면 됩니다

오늘 만날 문제 4

아래 두 개는 에러가 안 뜸

SyntaxError: await is only valid in async	원인 함수에 async 를 안 붙임	해결 function 앞에 async
Promise { pending } 이 찍힘	원인 await 를 빼먹음	해결 18장 참고. 앞에 await
undefined 가 나옴	원인 json 을 안 했거나 키 이름이 다름	해결 data 를 통째로 찍어보고 키를 확인
TypeError: Failed to fetch	원인 인터넷 또는 주소 형식 문제	해결 주소를 브라우저에 직접 넣어볼 것

로딩과 에러까지 붙이기

```
// 1. 불러오는 동안 불러오는 중 이라고 표시
// 2. 다 되면 그 문구를 지우기
// 3. 실패하면 문제가 생겼습니다 와 이유를 표시
// 4. 주소를 틀리게 해서 3번이 실제로 뜨는지 확인
```

힌트 01

36장 코드를 그대로 옮겨 쓰셔도 됩니다

힌트 02

로딩이 너무 빨라 안 보이면 앞에 `await wait(1000)` 을 넣어볼 것

남은 시간

15:00

오늘 뭐 입지

오늘 뭐 입지

지금 기온에 맞는 옷차림

서울

부산

대구

강릉

제주

제주

28.4도

민소매, 반팔, 반바지

습도 81퍼센트

시간대별

00:00
27.3도

01:00
27.2도

02:00
26.9도

03:00
26.6도

04:00
26.6도

05:00
26.7도

06:00
26.6도

07:00
27.0도

완성 예시, 도시를 바꾸면 다시 불러옴

■ 기본 요구사항

- 현재 기온을 받아와 화면에 표시
- 기온 구간에 맞는 옷차림 추천 문구 표시
- 불러오는 중과 실패 안내 표시

■ 심화 요구사항

- 도시를 골라 바꿀 수 있게
- 시간대별 예보를 목록으로 표시

한 번에 다 만들지 말 것

단계마다 화면을 확인하고 다음으로

1단계	기온을 받아서 콘솔에 찍기	fetch
2단계	화면에 기온과 추천 문구 표시	조건문
3단계	로딩과 실패 안내 붙이기	try catch
4단계	도시 선택과 시간대별 예보	map

전원 목표

2단계까지. 3단계부터는 여유가 있을 때

스타터 파일 구조

weather.html

```
1  const CITIES = [ /* 도시와 좌표 */ ];
2  function buildUrl(city) { /* 주소 만들기 */ }
3
4  // TODO 1  기온 받아오기
5  // TODO 2  기온으로 옷차림 정하기
6  // TODO 3  로딩과 실패 안내
7
8  function render(...) { // 화면 그리기, 완성됨 }
9  // 버튼 연결도 완성되어 있음
```

손대지 말 것

여기만 채우면 됨

다 한 사람은

기다리지 말고 추가문제로

01 최저 기온과 최고 기온 함께 보여주기

02 비 올 확률을 받아 우산 안내 붙이기

03 기온에 따라 배경색이 바뀌게

04 여러 도시를 한 번에 받아 나란히 보여주기

자 세 한 내 용 추가문제.md 에 있습니다. 하나만 해도 충분합니다

인터넷이 꿈긴 곳에서는 아무것도 못 보여 줄까

한 번 받아온 것을 어딘가 두면 되지 않을까. 답은 내일

오늘 배운 것

- 01 코드가 기다리지 않고 넘어간다는 것을 알게 됨
- 02 `async` 와 `await` 로 순서대로 읽히는 코드를 씀
- 03 `fetch` 로 진짜 데이터를 받아 화면에 그림

내일 예고 받아온 것을 남겨두는 법, 그리고 AI 가 짠 코드 고치기